

category: dotnet-general

tags: [newtonsoft-json, deserialization, constructor, JsonConvert, JObject]

severity: high

created: 2026-07-28

updated: 2026-07-28

project-origin: App-Access-V2 (iAccess Avalonia)

Newtonsoft.Json ném exception khi deserialize class có nhiều constructor không mặc định

Tình huống gặp phải

Port tầng thiết bị KZTEK (STEP-3.1 → 4.1), controller thật trả JSON `RegisterUserResult` qua `RegisterUserAsync()`, `AccessControllerClient.cs` gọi `JsonConvert.DeserializeObject<RegisterUserResult>(resultJson)`.

Model `RegisterUserResult` (trong assembly bất khả xâm phạm `IAccessv2.Objects`) có:

- Không có parameterless constructor
- 2 constructor có tham số:
 1. `(string cmdId, string deviceId, string accessKeyId, string customerId, string description, EmResgisterUserCode resultCode, int userId)` — 7 tham số
 2. `(RegisterUserControllerRequest request, string deviceId)` — 2 tham số, delegate tới constructor 1

Triệu chứng / Lỗi

```
Newtonsoft.Json.JsonSerializationException: Unable to find a constructor to use for type IAccessv2.Objects.Objects.Bases.RegisterUserResult. A class should either have a default constructor, one constructor with arguments or a constructor that is marked with the JsonConvert attribute.
```

Xảy ra ngay khi controller KZTEK thật (Phase 3+) trả JSON về — không phát hiện được qua build, chỉ crash lúc runtime khi có lệnh RegisterUser thật.

Nguyên nhân gốc rễ (Root Cause)

Newtonsoft.Json chọn constructor theo thứ ưu tiên:

1. Constructor có `[JsonConstructor]` attribute
2. Parameterless constructor
3. Nếu CHỈ có 1 constructor có tham số → thử match tên tham số với JSON property
4. Nếu có ≥ 2 constructor có tham số và không có `[JsonConstructor]` → THROW

Dù class có **public property setters** (set được sau khi tạo object), Newtonsoft vẫn phải khởi tạo object trước — nếu không xác định được constructor nào để gọi → exception.

Giải pháp

Option A — JObject.Parse + map thủ công (dùng khi model bất khả xâm phạm)

```
// Thay vì:  
// var result = JsonConvert.DeserializeObject<RegisterUserResult>(resultJson); // THROW  
  
// Dùng:  
var jobj = JObject.Parse(resultJson);  
var result = new RegisterUserResult(  
    jobj.Value<string>("CmdId") ?? string.Empty,  
    jobj.Value<string>("DeviceId") ?? string.Empty,  
    jobj.Value<string>("AccessKeyId") ?? string.Empty,  
    jobj.Value<string>("CustomerId") ?? string.Empty,  
    jobj.Value<string>("Description") ?? string.Empty,  
    (EmResgisterUserCode) (jobj.Value<int?>("ResultCode") ??  
    (int)EmResgisterUserCode.SUCCESS),  
    jobj.Value<int?>("UserId") ?? 0);
```

- Bọc trong **try/catch** để fallback khi JSON dị thường.
- Enum serialize mặc định là **int** (Newtonsoft default) → **Value<int?>** + cast.

Option B — [JsonConstructor] (dùng khi sửa được model)

```
[JsonConstructor] // Đánh dấu đúng 1 constructor, Newtonsoft match theo tên tham số  
public RegisterUserResult(string cmdId, string deviceId, ...)  
{...}
```

Không áp dụng được khi model trong assembly bất khả xâm phạm.

Áp dụng lại (How to reuse)

- Khi thấy **DeserializeObject<T>** trên bất kỳ type nào → kiểm tra xem T có parameterless constructor không (**grep "class T"** hoặc xem file source).
- Nếu type thuộc assembly legacy/bất khả xâm phạm không thể sửa → dùng JObject.Parse + map thủ công ngay từ đầu.
- Ưu tiên map thủ công khi JSON shape đơn giản (< 10 field) — dễ đọc, tường minh, không cần reflection.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ **Bug chỉ lộ khi controller thật trả JSON** — khi stub/NullDeviceController trả **null**, nhánh **if (!string.IsNullOrEmpty(resultJson))** không chạy nên không phát sinh crash. Rất khó phát hiện qua build hoặc test với stub.
- ⚠ **Enum serialize mặc định là int**, không phải string. Nếu cấu hình global converter đổi sang string thì **Value<int?>** sẽ fail — kiểm tra **JsonSerializerSettings** tại điểm gọi.
- ⚠ **JObject.Value<string>()** trả **null** nếu key không tồn tại (không throw) → cần **?? string.Empty** cho string, **?? 0** cho int.
- ⚠ **JObject.Parse** throws nếu **resultJson** là JSON không hợp lệ (không phải **null**) — cần bọc try/catch ở caller.

Tham chiếu

- Newtonsoft.Json docs:
[JsonConstructorAttribute](https://www.newtonsoft.com/json/help/html/JsonConstructorAttribute.htm)
- Project liên quan: App-Access-V2 — `iAccess.Core/Devices/AccessControllerClient.cs`
(STEP-4.1 G006 fix)
- File model:
`iAccessDesktopv2/IAccessv2.Objects/Objects/Bases/RegisterUserResult.cs`